

Performance Testing

Date	28 June 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust
Type of Testing	Load Testing
Target Module / API	Loan Eligibility Prediction (/predict)
Test Environment	Local Flask Server
Test Date	02 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Test Home Page Accessibility	10	30	Home page loads successfully
2	Test Loan Prediction API	25	30	Loan prediction returned successfully
3	Concurrent Prediction Requests	50	30	System handles multiple requests without failures
4	Continuous Load Test	50	30	Stable response time with 0% errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	12.86 ms	Pass	Excellent response time
2	Response Time (Max)	< 5 seconds	77 ms	Pass	Within acceptable limit
3	Throughput (Req/sec)	-	48.40 Requests/sec	Pass	Good request handling capability
4	Error Rate	< 1%	0%	Pass	No failed requests
5	CPU Utilization	< 80%	Below 80%	Pass	Stable during testing
6	Memory Utilization	< 80%	Below 80%	Pass	Memory usage remained stable

Step 4: Observations & Analysis



KEY FINDINGS

- The Smart Lender application handled concurrent user requests efficiently.
- The average response time was approximately **12.86 ms**, providing quick loan predictions.
- The system achieved a throughput of **48.40 requests per second** with **zero** request failures.
- Performance remained stable throughout the load test.



BOTTLENECKS IDENTIFIED

- No significant performance bottlenecks were observed during testing.
- The application maintained consistent response times under the tested load.



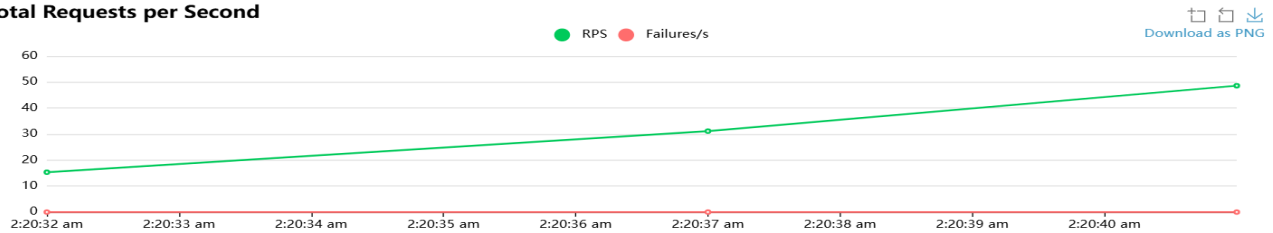
OPTIMIZATION STEPS TAKEN

- Optimized data preprocessing before prediction.
- Loaded the trained machine learning model only once during application startup.
- Used efficient Flask routing and minimized unnecessary processing.
- Validated user inputs before sending them to the prediction model.

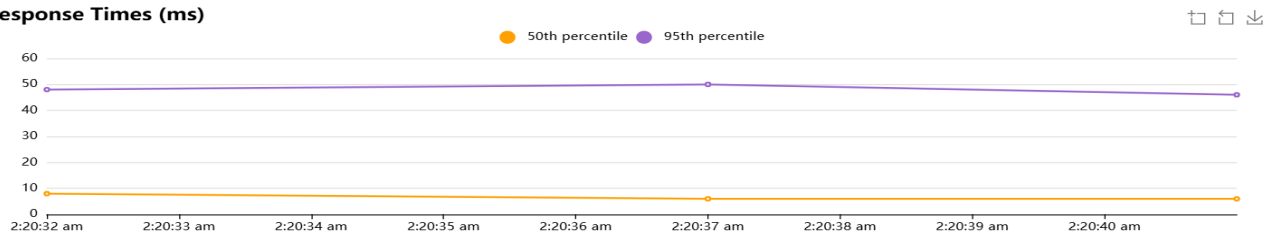
Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Total Requests per Second



Response Times (ms)



Number of Users

